

初学者入门手写数字识别案例

文档版本：V2.0

更新时间：2026 年 1 月 14 日

适用型号

序列	文档版本	适用型号	更新说明
1	1.0	SC171 开发套件 V3	使用 2.24 版本工具链
2	2.0	SC171 开发套件 V3	使用 2.29 版本工具链

目录

1 引言	1
2 数字识别模型搭建	1
2.1 数据准备	1
2.2 搭建神经网络	1
2.3 训练网络模型	2
2.4 模型测试	2
2.5 模型保存	3
3 Fibo AI Stack 模型转化	3
3.1 模型导入	3
3.2 模型转化	4
3.3 模型导出	5
4 数字识别模型部署	5
4.1 准备	5
4.2 Python sample 代码	6
4.3 模型的运行推理	8
5 Q&A	9
5.1 无法找到文件	9
5.2 环境配置失败	9
5.3 推理结果不准确	9

1 引言

本指南以手写数字识别为例，详细演示如何在 SC171 开发套件 V3 上通过 Fibocom AI Stack 实现高效的深度学习模型推理。通过结合专为边缘设备优化的 Fibo AI Stack 推理框架，本指南提供了一套完整的解决方案，覆盖从模型搭建、模型训练、模型推理到结果的全流程，旨在为边缘端 AI 部署提供标准化范例。

2 数字识别模型搭建

以下显示黑色背景的代码均为在电脑端的 LeNet.py

2.1 数据准备

- 从 Keras 库中引入手写数字数据集 MNIST，他是一个包含 60000 个训练样本和 10000 个测试样本的数据集。使用 load_data() 函数将 MNIST 数据集加载到程序中，并将数据集分为训练集和测试集，其中 train_images、train_labels 为训练集，test_images、test_labels 为测试集。
- 由于神经网络只接受数值型数据，所以需要将由手写数字图像转换为数据张量格式。也就是将每张图像转换为 28×28 的矩阵，并进行归一化处理。

```
from keras.utils import to_categorical
from keras import models, layers
from keras.optimizers import RMSprop
from keras.datasets import mnist
import tensorflow as tf

# 加载数据集
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

# 数据处理
train_images = train_images.reshape((60000, 28, 28, 1)).astype('float') / 255
test_images = test_images.reshape((10000, 28, 28, 1)).astype('float') / 255
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
```

2.2 搭建神经网络

模型采用了 Sequential 模型

- C1 层是卷积层，包含 6 个特征图，是由 6 个 3×3 的卷积核对输入图像卷积得到。
- S2 层是一个下采样层，包含 6 个特征图，是由 C1 层的特征图经过 2×2 的窗口进行平均池化
- C3 层是卷积层，包含 16 个特征图，是由 16 个 3×3 的卷积核对 S2 进行卷积得到
- S4 是一个下采样层，包含 16 个特征图，是由 C3 层的特征图经过 2×2 的窗口进行平均池化

- C5 是卷积层包含 120 个特征图，是由 120 个 3x3 的卷积核对 S4 进行卷积得到。
- F6 是包含 84 个神经元的全连接层，采用 relu 激活函数
- 最后添加一个分类层，使用 softmax 激活。输出 0-9 是个数字，所以单元数为 10

模型搭建完成后，进行编译模型。使用交叉熵作为损失函数，RMSprop 优化器进行训练，在训练过程中监测模型的精度。

```
# 搭建LeNet网络
def LeNet():
    network = models.Sequential()
    network.add(layers.Conv2D(filters=6, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)))
    network.add(layers.AveragePooling2D((2, 2)))
    network.add(layers.Conv2D(filters=16, kernel_size=(3, 3), activation='relu'))
    network.add(layers.AveragePooling2D((2, 2)))
    network.add(layers.Conv2D(filters=120, kernel_size=(3, 3), activation='relu'))
    network.add(layers.Flatten())
    network.add(layers.Dense(84, activation='relu'))
    network.add(layers.Dense(10, activation='softmax'))
    return network
network = LeNet()
network.compile(optimizer=RMSprop(Lr=0.001), Loss='categorical_crossentropy', metrics=['accuracy'])
```

2.3 训练网络模型

使用 fit() 方法对构建好的神经网络进行训练，epochs 表示训练多少个回合，batch_size 表示每次训练给多大的数据。

```
# 训练网络，用fit函数，epochs表示训练多少个回合，batch_size表示每次训练给多大的数据
network.fit(train_images, train_labels, epochs=20, batch_size=128, verbose=2)
```

2.4 模型测试

使用 evaluate() 方法对模型进行测试，并返回测试误差和测试准确率。

```
# 在测试集上测试一下模型的性能
test_loss, test_accuracy = network.evaluate(test_images, test_labels)
print("test_loss:", test_loss, "test_accuracy:", test_accuracy)
```

下面为模型测试的结果

```
Epoch 16/20
469/469 - 6s - loss: 0.0077 - accuracy: 0.9976 - 6s/epoch - 13ms/step
Epoch 17/20
469/469 - 6s - loss: 0.0076 - accuracy: 0.9976 - 6s/epoch - 13ms/step
Epoch 18/20
469/469 - 6s - loss: 0.0070 - accuracy: 0.9976 - 6s/epoch - 13ms/step
Epoch 19/20
469/469 - 6s - loss: 0.0061 - accuracy: 0.9982 - 6s/epoch - 13ms/step
Epoch 20/20
469/469 - 7s - loss: 0.0051 - accuracy: 0.9984 - 7s/epoch - 14ms/step
313/313 [=====] - 1s 2ms/step - loss: 0.0474 - accuracy: 0.9890
test_loss: 0.04737924039363861 test_accuracy: 0.9890000224113464
```

2.5 模型保存

Keras 支持保存 HDF5 文件，也就是.h5 文件，这个文件包含模型的体系结构，权重值和 compile() 信息。

因为 Fibo AI Stack 模型转化工具支持 onnx-dlc、tflite-dlc 与 TensorFlow-dlc，当前不支持直接将.h5 文件转化成 dlc，所以说需要先将.h5 文件转化成 tflite，然后在用 Fibo AI Stack 模型转化工具将 tflite 转化成 dlc。

```
# 模型保存为.h5格式
network.save('MyModel.h5')
# 模型转换为.tflite格式
keras_file = 'MyModel.h5'
tf.keras.models.save_model(network, keras_file)
model = tf.keras.models.load_model(keras_file)
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()
with open('MyModel.tflite', 'wb') as f:
    f.write(tflite_model)
```

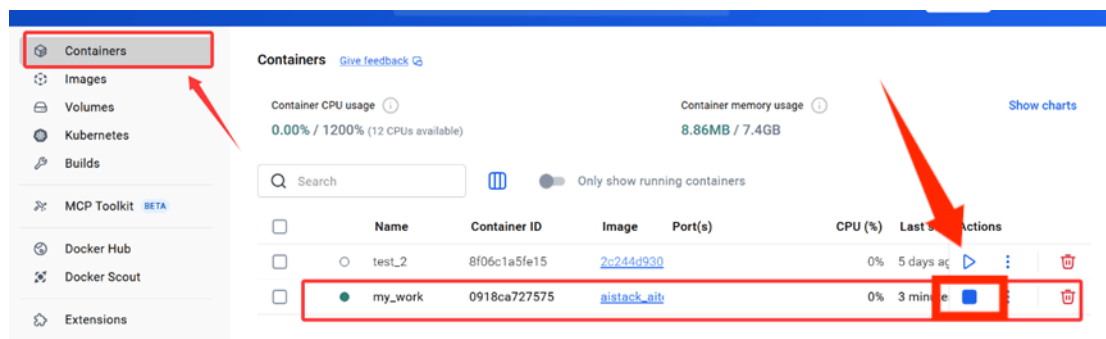
在电脑端运行 LeNet.py 后，工程文件夹中会出现 MyModel.h5 和 MyModel.tflite 两个文件。

MyModel.h5	2023/10/20 9:49	H5 文件	902 KB
MyModel.tflite	2023/10/20 9:49	TFLITE 文件	434 KB

3 Fibo AI Stack 模型转化

3.1 模型导入

打开 Docker Desktop，点击运行虚拟机并打开终端。（具体环境搭建操作可在 AI 端侧部署开发中的课程查看，课程名称为“Fibo AI Stack 模型转化指南”）



在终端输入命令，查看正在运行的虚拟机，如图所示即可：`docker ps`

```
PS C:\Users\FIBOCOM> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS   NAMES
0918ca727575   aistack_aitools_229_env:latest     "bash"                  8 minutes ago Up 8 minutes   my_work
```

输入以下命令，将电脑中的模型文件传输到虚拟机中：

`docker cp <电脑中的模型文件路径> <虚拟机名称>:<虚拟机中的文件路径>`

如：

`docker cp D:\Project\MyModel.tflite my_work:/project/`

如图所示，传输成功

注意：要在**电脑终端**输入该命令

```
PS C:\Users\FIBOCOM> docker cp "D:\Project\MyModel.tflite" my_work:/project/
Successfully copied 445kB to my_work:/project/
PS C:\Users\FIBOCOM>
```

3.2 模型转化

输入命令，进入虚拟机终端中：`docker exec -it <虚拟机名称> bash`

如：`docker exec -it my_work bash`

```
PS C:\Users\FIBOCOM> docker exec -it my_work bash
[INFO] AISW SDK environment set
[INFO] QNN_SDK_ROOT: /opt/2.29.0.241129
[INFO] SNPE_ROOT: /opt/2.29.0.241129
(python310_env) root@0918ca727575:/#
```

使用 Fibo AI SDK 中的“snpe-tflite-to-dlc”工具，将 tflite 格式模型转化为 DLC 格式模型，具体方法：

`snpe-tflite-to-dlc --input_network /路径/model.tflite --input_dim input_name "1,299,299,3" --output_path /路径/model.dlc`

如：

```
snpe-tflite-to-dlc --input_network /home/project/MyModel.tflite --input_dim
"serving_default_conv2d_input:0" "1,28,28,1" --output_path
/home/project/MyModel.dlc
```

注释：

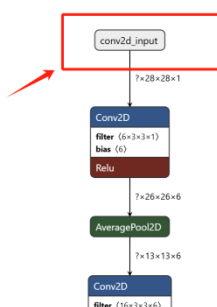
--input_network 参数表示：需要转换的模型框架路径

--output_path 参数表示：转换模型文件输出路径

--input_dim 参数表示：需转化模型的输入名称和输入数据格式，对于输入名称和输入数据格式不清楚的用户，可以在 <https://netron.app/> 中导入自己的模型，即可查看到输入名称和输入数据格式

红色字体部分需自行填入

下面进行示例



name	main
signature	serving_default
INPUTS	
模型输入名称	
conv2d_input	name: serving_default_conv2d_input:0
	tensor: float32[1,28,28,1]
	identifier: 0
OUTPUTS	
模型输入数据格式	
dense_1	name: StatefulPartitionedCall:0
	tensor: float32[1,10]
	identifier: 20



如图所示，输入命令，转化成功

```
2025-04-07 08:16:46,513 - 235 INFO - INFO_INITIALIZATION_SUCCESS:
2025-04-07 08:16:46,518 - 235 INFO - INFO_CONVERSION_SUCCESS: Conversion completed successfully
2025-04-07 08:16:46,526 - 235 INFO - INFO_WRITE_SUCCESS:
```

3.3 模型导出

输入命令退出虚拟机终端：`exit`

输入命令将文件导出虚拟机：`docker cp <虚拟机名称>:<虚拟机中的模型路径> <电脑中的模型文件路径>`

如：`docker cp my_work:/project/MyModel.dlc D:\Project\MyModel.dlc`

如图所示，文件导出成功

名称	修改日期	类型	大小
deeplabv3_resnet50-snapdragon_8_elite.dlc	2025/3/25 16:35	文件夹	
deeplabv3_resnet50-snapdragon_8_elite.tflite	2025/3/25 15:58	TFLITE 文件	154,762 KB
MyModel.dlc	2025/4/7 16:16	DLC 文件	445 KB

4 数字识别模型部署

4.1 准备

1. 将转换后的 dlc 文件以及所需的其他文件放入 SC171 开发套件 V3 开发板中，注意：要将本案例附件中的推理文件 `number.py`、`Final_number.py` 与 `api_infer.py` 放在同一路径下。

2. 导入所需的库，在终端输入以下命令：

```
pip3 install numpy opencv-python
```


3. 需要准备的参数:

- dlc_path: 模型文件路径 (.dlc 文件)
- img_path: 输入图片路径
- Label_file: 标签文件路径
- 模型相关数据: 输入张量名称、输出张量名称

注:

- number.py 文件用于推理本地照片
- camera.py 用于测试摄像头 (可根据命令查询个人摄像头信息 ID)
- Final_number.py 文件使用摄像头拍照后, 进行手写数字识别推理

4.2 Python sample 代码

以下显示黑色背景的代码均为在开发板端上的 number.py

1. 调用库, 定义文件路径
2. 模型节点相关配置
3. 配置推理引擎配置字典, 更换推理设备 (如 CPU 或 GPU)。

```
import cv2
import numpy as np
from api_infer import * # Fibo AI推理SDK接口

# 配置区 (用户需修改部分)
model_path = "/home/fibo/project/number/MyModel.dlc"
input_path = "/home/fibo/project/number/image1.jpg"
save_path = "/home/fibo/project/number/"
label_file = "/home/fibo/project/number/label.txt"

# 模型节点配置 (根据实际模型修改)
INPUT_NAME = "serving_default_conv2d_input:0"
OUTPUT_NAME = "StatefulPartitionedCall:0"

# 推理引擎配置字典
inference_config = {
    'model': model_path,
    'platform': "qualcomm",
    'framework': "snpe",
    'runtime': "GPU",
    'log_level': "INFO",
    'profile_level': 0,
}
```

4. 使用到的功能函数, 以及根据使用的模型需求来更改对输入内容的前处理和后处理的代码部分。

```
# 标签读取函数
def read_label_list(label_path):
    """读取标签文件"""
    with open(label_path, 'r', encoding="utf-8") as f:
        return f.read().splitlines()

# 图像预处理函数
def preprocess(image_path):
    """图像预处理流程"""
    # 读取图片
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f"无法读取图片: {image_path}")
    # 灰度图
    gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # 反色处理
    img_inverted = 255 - gray_img
    # 二值化
    _, binary_img = cv2.threshold(img_inverted, 127, 255, cv2.THRESH_BINARY)
    # 调整大小
    resized_img = cv2.resize(binary_img, (28, 28), interpolation=cv2.INTER_AREA)
    # 再次二值化
    _, final_img = cv2.threshold(resized_img, 20, 255, cv2.THRESH_BINARY)
    # 保存预处理图片
    cv2.imwrite(f"{save_path}/preprocessed_image.jpg", final_img)
    # 转换为模型输入格式
    final_img_normalized = final_img.astype(np.float32) / 255.0
    final_img_flattened = final_img_normalized.flatten()

    return final_img_flattened

# 结果后处理函数
def postprocess(outputs, label_list):
    """推理结果后处理"""
    if OUTPUT_NAME not in outputs:
        print(f"错误: 找不到输出节点 '{OUTPUT_NAME}'")
        return None
    output_data = outputs[OUTPUT_NAME]
    output_array = np.array(output_data)
    predicted_class_id = np.argmax(output_array)
    confidence = float(np.max(output_array))
    class_name = label_list[predicted_class_id] if predicted_class_id < len(label_list) else f"类别{predicted_class_id}"
    result = {
        "class_id": int(predicted_class_id),
        "class_name": class_name,
        "confidence": confidence,
        "all_scores": output_array.tolist()
    }
    print(f"预测结果: {class_name} (置信度: {confidence:.2%})")
    return result
```

5. 主执行流程

```
4     return result
5
6 # 主执行流程
7 if __name__ == "__main__":
8     print("初始化推理引擎...")
9     # 创建推理会话, 初始化推理引擎
10    inference_session = InferenceSession(**inference_config)
11    assert inference_session.Initialize() == 0, "推理引擎初始化失败"
12
13    # 读取标签
14    label_list = read_label_list(label_file)
15
16    # 执行预处理
17    input_data = preprocess(input_path)
18
19    # 准备输入数据
20    input_feed = {INPUT_NAME: input_data}
21    # 输出层名称列表 (空列表表示获取所有输出)
22    output_names = [OUTPUT_NAME] if OUTPUT_NAME else []
23    # 执行推理, 返回包含输出结果的字典
24    print("执行推理...")
25    outputs = inference_session.Execute(output_names, input_feed)
26    assert outputs is not None, "推理失败"
27
28    # 执行后处理
29    result = postprocess(outputs, label_list)
30
31    # 释放资源
32    assert inference_session.Release() == 0, "资源释放失败"
33
34    print("推理完成")
```

输入数据前处理

推理部分

输出数据后处理

4.3 模型的运行推理

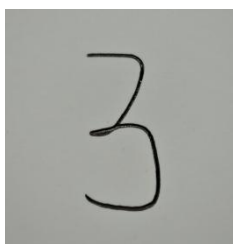
调用 sample 执行文件进行推理, 在开发板终端输入以下命令:

```
python3 <Python sample 文件地址>
```

如:

```
python3 /home/fibo/number/number.py
```

输入内容, 如图所示



输出结果, 执行成功, 结果如图显示: 3

```
[2025-12-15 19:54:05.639] [Fiboaisdk] [---I---] [thread 18753] SNPE Major Version: 2
[2025-12-15 19:54:05.639] [Fiboaisdk] [---I---] [thread 18753] SNPE Minor Version: 29
[2025-12-15 19:54:05.639] [Fiboaisdk] [---I---] [thread 18753] SNPE Teeny Version: 0
[2025-12-15 19:54:05.639] [Fiboaisdk] [---I---] [thread 18753] SNPE Version String: 2.29.0.241129103708 105762
[2025-12-15 19:54:05.640] [Fiboaisdk] [---W---] [thread 18753] Device Init failed, libfiq sysmon.so open lib failed
hogl::output file: failed open file /home/fibo/project/number/Log/DebugLog.10.txt. Permission denied (13)
[USER ERROR] Failed to create log file outputter: Could not initialize file outputter with path: /home/fibo/project/number/Log/DebugLog.#.txt
5Failed to start Logger!
[2025-12-15 19:54:05.823] [Fiboaisdk] [---I---] [thread 18753] [registry] [snpe] Set gpu_float32_16 hybrid runtime succeeded
[2025-12-15 19:54:05.823] [Fiboaisdk] [---I---] [thread 18753] [registry] [snpe] Set cpu_float32 runtime succeeded
[2025-12-15 19:54:06.515] [Fiboaisdk] [---I---] [thread 18753] [registry] [snpe] Set gpu_float32_16 hybrid runtime succeeded
[2025-12-15 19:54:06.515] [Fiboaisdk] [---I---] [thread 18753] [registry] [snpe] Set cpu_float32 runtime succeeded
执行推理...
[INFO] input type: float32
[2025-12-15 19:54:06.572] [Fiboaisdk] [---I---] [thread 18753] [registry] [snpe] [multiple] SnpeInferPriv::Process whole time: 2859 us
预测结果: 3 (置信度: 99.95%)
推理完成
```

5 Q&A

5.1 无法找到文件

可能原因:

1. 模型、标签等文件路径配置错误。
2. 模型文件未正确下载或解压。
3. 无权限访问文件。

解决办法:

1. 修改模型的存放路径。
2. 确保模型文件已正确下载并解压到指定路径。
3. 确保运行的 Python 文件路径正确。
4. 检查文件权限，确保当前用户有权限访问模型文件。

5.2 环境配置失败

可能原因:

缺少必要的 Python 包。

解决办法:

安装必要的 Python 包。

5.3 推理结果不准确

可能原因:

1. 输入图像分辨率不符合模型要求。
2. 模型未正确加载或配置。

解决办法:

1. 确保输入图像格式正确使用，可以输出输入数据来检查格式是否符合模型要求：`print(input_feed)`
2. 检查模型，输入输出 Tensor 名称等信息正确。
3. 重新加载模型并运行推理，确保模型配置无误。
4. 检查输入图像的预处理步骤，确保图像数据格式符合模型要求。